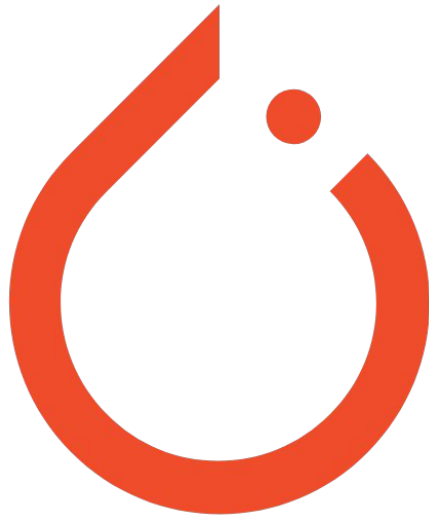


From Python to FXGraph: How TorchDynamo Captures PyTorch Programs



By: **Ayush Satyam**
Associate MLE at  **Red Hat**

what is TorchDynamo?

It's a Python-level JIT compiler that captures PyTorch programs into FX graphs by symbolically executing Python bytecode.

How it does that?

It hooks into CPython's PEP 523 frame evaluation API to intercept execution, traces operations into an FX graph

CPython prerequisites??

Some CPython internal workings are required to fully capture the essence of Dynamo and how it intercepts the frames.

Source text: "def add(a, b):\n result = a + b\n return result"



Parser + Tokenizer



AST: FunctionDef → Assign(BinOp(a + b)) → Return(result)



Symbol Table: a=LOCAL, b=LOCAL, result=LOCAL



Compiler walks AST, emits instructions per node:

Name('a') → LOAD_FAST 0

Name('b') → LOAD_FAST 1

Add → BINARY_ADD

Assign → STORE_FAST 2

Name('result') → LOAD_FAST 2

Return → RETURN_VALUE



Assembler: instructions → bytes, compute jumps, compute stack size



PyCodeObject:

co_code = b'|\x00|\x01\x17\x00}\x02|\x02S\x00'

co_varnames = ('a', 'b', 'result')

co_stacksize = 2

...

Thread State (PyThreadState)

- has a pointer to the interpreter state
- interpreter state has eval_frame function pointer



_PyEval_EvalCode()

- receives: code object for add, globals dict, args (3, 5)
- creates PyFrameObject:
 - f_code = add's code object
 - f_localsplus = [PyLong(3), PyLong(5), NULL, <stack space>...]
 - f_valustack = &f_localsplus[3] (stack starts after locals)
 - f_back = the <module> frame
 - f_lasti = -1



PyEval_EvalFrameEx(frame)

- calls: tstate->interp->eval_frame(tstate, frame, 0)
- normally this is _PyEval_EvalFrameDefault
- BUT Dynamo replaces it with dynamo__custom_eval_frame



_PyEval_EvalFrameDefault(tstate, frame, 0)

- sets up: co, first_instr, next_instr, stack_pointer, fastlocals
- enters the big for(;;) loop:



- LOAD_FAST 0 → push fastlocals[0] (3)
- LOAD_FAST 1 → push fastlocals[1] (5)
- BINARY_ADD → pop 5, peek 3, call PyNumber_Add → push 8
- STORE_FAST 2 → pop 8, store in fastlocals[2]
- LOAD_FAST 2 → push fastlocals[2] (8)
- RETURN_VALUE → pop 8, return it

Entrypoints to Dynamo

CPython internals

Every function call in CPython:

`PyEval_EvalFrameEx(frame)`

↳ `tstate->interp->eval_frame(frame)`

normally: `_PyEval_EvalFrameDefault`
with Dynamo: `dynamo__custom_eval_frame`

Frame is
intercepted here

Python API

`torch.compile(fn)`

↓
sets up callback

↓
Dynamo REPLACES this
with its own function



Let's see an example

```
import torch

def fn(x, y):
    z = x + y
    return torch.relu(z)

compiled_fn = torch.compile(fn)
compiled_fn(torch.randn(4), torch.randn(4))
```

Python bytecode

```
LOAD_FAST      0 (x)
LOAD_FAST      1 (y)
BINARY_ADD
STORE_FAST     2 (z)
LOAD_GLOBAL     0 (torch)
LOAD_ATTR      1 (relu)
LOAD_FAST      2 (z)
CALL_FUNCTION   1
RETURN_VALUE
```

Frame object for the function

Frame Object for fn(x, y):

f_code → fn's code object (the bytecode above)

f_localsplus → [x_tensor, y_tensor, NULL, <stack space...>]

slot 0 slot 1 slot 2 ← value stack starts here

f_globals → module's global dict

f_back → the caller's frame

Entrypoint: The Hook

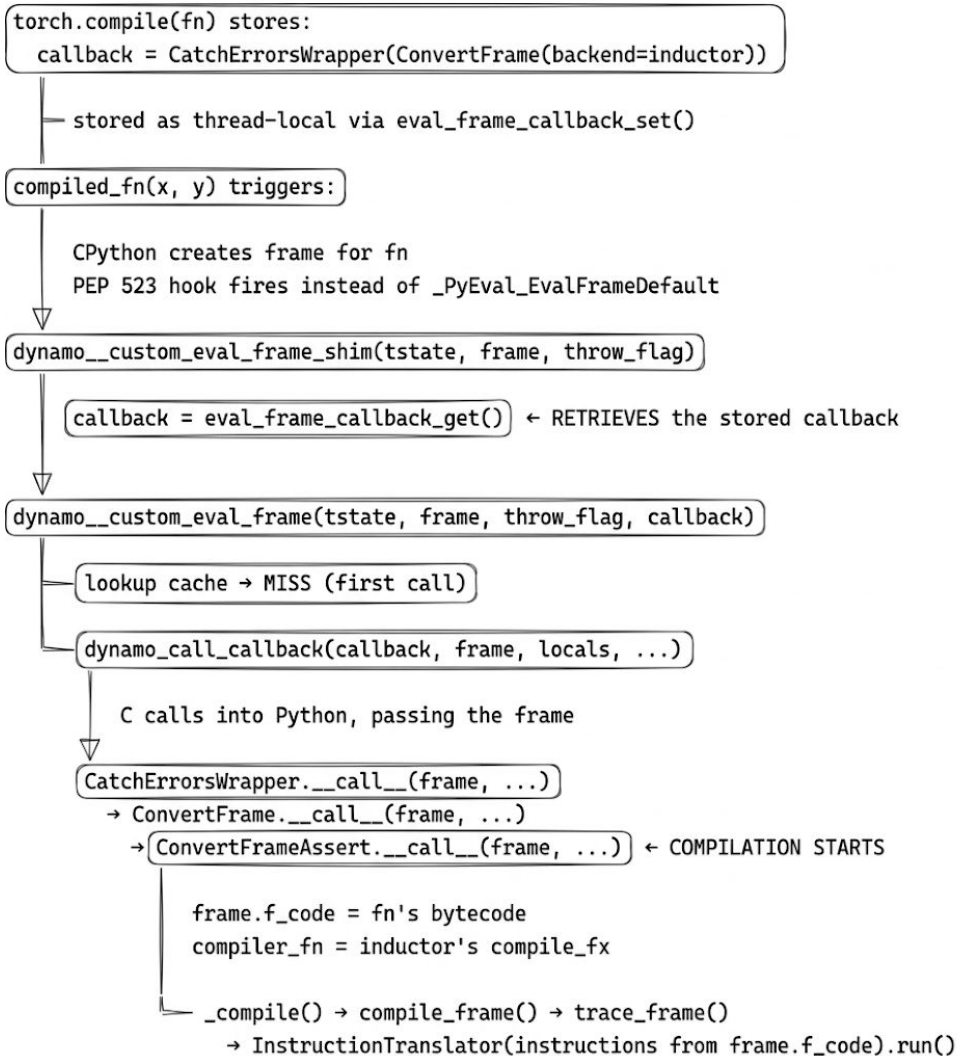
```
// torch/csrc/dynamo/eval_frame

static void enable_eval_frame_shim(PyThreadState* tstate) {
    previous_eval_frame = _PyInterpreterState_GetEvalFrameFunc(tstate->interp);
    _PyInterpreterState_SetEvalFrameFunc(
        tstate->interp, &dynamo_custom_eval_frame_shim);
}
```

Execution path of torch.compile

```
# torch/__init__.py – torch.compile delegates to:  
torch._dynamo.optimize(backend="inductor")  
  
# eval_frame.py – optimize creates the callback:  
return _optimize_catch_errors(  
    convert_frame.convert_frame(backend, hooks),  # ← THE CALLBACK  
    ...  
)
```

Where do the two paths combine?



The Python callback does three things

```
ConvertFrameAssert.__call__(frame)
↳ _compile(frame.f_code, ...)
    ↳ compile_frame(code, ...)
        ↳ transform_code_object(code, transform)
```

① DISASSEMBLE: `code.co_code` → list of Instruction objects
(same `LOAD_FAST`, ...)

② TRACE: `trace_frame(code, ...)` → `InstructionTranslator.run()`
`graph = ...`
(symbolic execution ...)

③ REASSEMBLE: modified instructions → new `CodeType` object
(the NEW bytecode ...)

Returns: new code object + guards

The InstructionTranslator replays the same bytecode....but symbolically

```
# symbolic_convert.py
def run(self):
    while self.step():    # ← same loop as CPython's for(;;)
        pass

# symbolic_convert.py
def step(self):
    inst = self.instructions[ip]
    self.dispatch_table[inst.opcode](self, inst)  # ← same as switch(opcode)
```

Dynamo Symbolic execution

Instruction	CPython (real)	Dynamo (symbolic)
LOAD_FAST 0 (x)	PUSH(PyTensor*)	push(TensorVariable(proxy=arg_0))
LOAD_FAST 1 (y)	PUSH(PyTensor*)	push(TensorVariable(proxy=arg_1))
BINARY_ADD	PyNumber_Add → real tensor	TensorVariable.call_function() → creates FX node: add(arg_0, arg_1) push(TensorVariable(proxy=add))
STORE_FAST 2 (z)	fastlocals[2] = result	symbolic_locals['z'] = TensorVariable(proxy=add)
LOAD_GLOBAL (torch)	PUSH(torch module)	push(PythonModuleVariable(torch))
LOAD_ATTR (relu)	getattr(torch, 'relu')	push(TorchInGraphFunctionVariable(relu))
LOAD_FAST 2 (z)	PUSH(fastlocals[2])	push(TensorVariable(proxy=add))
CALL_FUNCTION 1	torch.relu(z) → real tensor	TorchInGraphFunctionVariable.call_function() → creates FX node: relu(add) push(TensorVariable(proxy=relu))
RETURN_VALUE	return real tensor	→ compile_subgraph() triggered! FX graph is finalized Backend is called

3 layers of InstructionTranslator

Layer 1

It symbolically executes
bytecode. When it hits a
CALL instruction (e.g.,
torch.add(x, y)), it:

- Pops VariableTrackers off
the symbolic stack
- Calls fn.call_function(self,
args, kwargs) on the
function variable

Layer 2

Each VariableTracker
subclass decides what
happens when it's "called."

```
tensor_variable = wrap_fx_proxy(  
    tx=tx,  
    proxy=tx.output.create_proxy(  
        "call_function",  
        fn_,  
        *proxy_args_kwargs(args, kwargs),  
    ),  
)
```

FX proxy object gets created
which converts to fx.node

Layer 3

When InstructionTranslator
hits RETURN_VALUE

- compile_subgraph() adds
an output node pointing
to whatever
TensorVariable is on top
of the symbolic stack
- GraphModule -- wraps the
fx.Graph into a callable
torch.fx.GraphModule
- Hand over to backend

What comes ahead?

- "What happens on the second call?" — Guards, the C++ guard tree, caching, shadow frames
- "What happens to the FX graph after Dynamo?" — AOTAutograd, partitioning, Inductor

THANK YOU